

Guide Fondamental Python : Syntaxe, Structures de Données et Regex

1. Origines, Syntaxe et Environnement

Repères chronologiques

- 1989-1991 : Créé par Guido van Rossum (inspiré d'ABC). Version 0.9.0 (fonctions, classes, modules).
- 1994 : Python 1.0 (exceptions, modules robustes).
- 2000 : Python 2.0 (gestion mémoire, compréhensions de listes).
- 2008-2020 : Python 3.0 (rupture de rétrocompatibilité). Fin de vie définitive de Python 2.7 en janvier 2020.
- 2023+ : Python 3.12 (performances accrues, PEP 701 pour les f-strings), préparation des t-strings (PEP 750, 3.14).

Règles de syntaxe fondamentales

- Indentation stricte : Délimite les blocs de code via des espaces/tabulations (remplace les accolades {}).
- Sensibilité à la casse : nom et Nom sont deux variables distinctes.
- Typage dynamique : Le type est déduit à l'assignation de la valeur, sans déclaration préalable.

Installation et scripts autonomes

- Gestionnaires de paquets :
 - Debian/Ubuntu : `sudo apt update && sudo apt install python3`
 - Red Hat/Fedora : `sudo dnf install python3`
 - Windows : `choco install python`
 - macOS : `brew install python@3`
- Vérification : `python --version` ou `python3 --version`.
- Shebang UNIX : Indique au shell quel interpréteur appeler :

Python

```
#!/usr/bin/env python3
```

```
# Auteur : Nom | Date : AAAA-MM-JJ | Description : Rôle du script
```

- Exécution directe sous Linux/macOS :

Bash

```
chmod +x mon_script.py
```

```
./mon_script.py
```

2. Variables, Types Primitifs et Opérateurs

Types de base et conversion

- Entier (int) : 42
- Flottant (float) : 3.14
- Chaîne (str) : "texte" ou 'texte'
- Booléen (bool) : True ou False (Attention : True == 1 et False == 0).
- Transtypage (Casting) : int("123"), float("3.14"), str(42).

Opérations arithmétiques et logiques

- Arithmétique : + (addition), - (soustraction), * (multiplication), / (division flottante, ex: $10 / 3 = 3.333...$), % (modulo : reste de la division).
- Priorité : Parenthèses prioritaires sur la multiplication/division, elles-mêmes prioritaires sur l'addition/soustraction.
- Logique booléenne :
 - and : Vrai uniquement si les deux opérandes sont vrais.
 - or : Vrai si au moins un opérande est vrai.
 - not : Inverse la valeur de vérité (not True donne False).

3. Chaînes de Caractères et Formatage

Les chaînes sont immuables. Toute opération produit une nouvelle chaîne en mémoire.

Manipulation et inspection

- Concaténation et longueur : s1 + " " + s2, len(s).
- Casse : .upper(), .lower(), .capitalize() (1re lettre), .title() (chaque mot).
- Nettoyage : .strip() (espaces aux deux extrémités), .lstrip(), .rstrip().
- Recherche : in (présence booléenne), .startswith(), .endswith(), .find() (index ou -1).
- Substitution : .replace("ancien", "nouveau", [max_occurrences]).
- Validation : .isdigit() (chiffres uniquement), .isalpha() (lettres uniquement), .isalnum(), .isspace().

- Découpage / Assemblage :
 - `chaine.split(sep)` : Découpe en liste (par défaut sur les espaces multiples).
 - `chaine.splitlines()` : Découpe sur les retours chariot.
 - `sep.join(liste_str)` : Fusionne une liste de chaînes (les éléments non-str doivent être convertis au préalable).
- Préfixes et suffixes stricts (Python 3.9+) :
 - `.removeprefix("pre_")` et `.removesuffix(".ext")` (contrairement à `.strip()` qui traite un ensemble de caractères individuels).
- Alignement : `.zfill(largeur)` (zéros à gauche), `.ljust(largeur)`, `.rjust(largeur)`.

Formatage des chaînes

- Historique : Opérateur % (ex: "%s a %d ans" % (nom, age)), méthode `.format()` (ex: "{} a {}".format(nom, age)).
- f-strings (f"...", Python 3.6+) :
 - Expressions et appels directs : `f"{largeur * hauteur}"`, `f"{nom.upper()}"`.
 - Débogage auto-documenté (Python 3.8+) : `f"{variable=}"` affiche `variable=valeur`.
 - Conversions explicites : `!r` (applique `repr()`), `!s` (applique `str()`).
 - Décimales : `f"{pi:.2f}"` -> 3.14.
 - Séparateur de milliers : `f"{nombre:,}"` -> 1,234,567,890.
 - Remplissage par zéros : `f"{id:04d}"` -> 0042.
 - Pourcentage : `f"{ratio:.1%}"` -> 75.6%.
 - Alignement : `f"{val:>10}"` (droite), `f"{val:<10}"` (gauche), `f"{val:^10}"` (centre).
 - Échappement : Doubler les accolades `{{` et `}}` pour afficher des accolades littérales.
 - Dates avec datetime : `f"{date:%d/%m/%Y %H:%M:%S}"`, `f"{date:%A, %d %B %Y}"`.
 - PEP 701 (Python 3.12) : Réutilisation autorisée des mêmes guillemets à l'intérieur des expressions.

- PEP 750 (Python 3.14, t-strings) : `t"..."` génère un objet Template pour retarder l'évaluation et assainir contextuellement les variables (protection contre les injections HTML/SQL).

4. Listes (list)

Collection ordonnée, indexée, hétérogène et mutable, délimitée par `[]`.

Création et accès

- Initialisation : `l = []`, `l = [1, "a", True, 3.14]`, ou `list(range(debut, fin, pas))`.
- Indexation : `l[0]` (premier), `l[-1]` (dernier), `l[-2]` (avant-dernier).

Modification et méthodes

- Ajout :
 - `.append(x)` : Ajoute `x` à la fin.
 - `.insert(i, x)` : Insère `x` à l'index `i`.
 - `.extend(iterable)` : Fusionne un autre itérable en place (opérateur `+` crée une nouvelle liste).
- Suppression :
 - `.remove(x)` : Supprime la première occurrence de la valeur `x`.
 - `.pop([i])` : Supprime et retourne l'élément à l'index `i` (par défaut le dernier).
 - `del l[i]` ou `del l[start:stop]` : Suppression par index ou plage.
- Inspection et réorganisation :
 - `len(l)` : Nombre d'éléments.
 - `l.index(x)` : Index de la première occurrence (lève `ValueError` si absent).
 - `x in l` : Test d'appartenance booléen ($\mathcal{O}(n)$).
 - `l.count(x)` : Nombre d'occurrences.
 - `l.sort(reverse=False)` : Trie en place. Fonction `sorted(l)` retourne une nouvelle liste triée.
 - `l.reverse()` : Inverse les éléments en place.

Itération et Compréhensions

- Parcours :
 - `for item in l:`

- `for i, item in enumerate(l):` (fournit le couple (index, valeur)).
- Compréhension de liste : `[expression for element in iterable if condition]`

Python

```
pairs = [x for x in ma_liste if x % 2 == 0]
```

Aplatir une liste imbriquée :

```
aplatie = [el for sous_liste in matrice for el in sous_liste]
```

5. Dictionnaires (dict)

Collection de paires clé: valeur. Mutable. Préserve l'ordre d'insertion (depuis Python 3.7).

- Règle des clés : Doivent être uniques et immuables/hachables (str, int, float, tuple). Les types mutables (list, dict) sont interdits.
- Collision de clés : True, 1 et 1.0 partagent le même hachage et représentent une seule et même clé.
- Complexité : Accès direct par clé en $\mathcal{O}(1)$ en moyenne, contre $\mathcal{O}(n)$ pour une liste.

Opérations fondamentales

- Création : `{}` (vide), `{"a": 1}`, `dict(a=1, b=2)`, ou `dict(zip(cles, valeurs))`.
- Accès :
 - `d[cle]` : Rapide, mais déclenche une `KeyError` si la clé est absente.
 - `d.get(cle, default=None)` : Sécurisé, renvoie None ou la valeur par défaut.
 - `cle in d` : Vérifie l'existence de la clé.
- Édition et mise à jour :
 - `d[cle] = valeur` : Crée la clé ou écrase sa valeur.
 - `d.update(autre_dict)` : Met à jour ou ajoute plusieurs paires.
 - `d.setdefault(cle, default)` : Ajoute la clé uniquement si elle n'existe pas déjà.
 - `d1 | d2` (crée un nouveau dictionnaire) et `d1 |= d2` (mise à jour en place) (Python 3.9+).
 - Dépaquetage : `**d1, **d2`.
- Suppression :

- `d.pop(cle, [default])` : Supprime et retourne la valeur (évite `KeyError` si un repli est fourni).
- `del d[cle]` : Supprime la paire.
- `d.popitem()` : Supprime et retourne la dernière paire (clé, valeur) insérée (LIFO).
- `d.clear()` : Vide le dictionnaire sans détruire l'objet en mémoire.

Parcours et Compréhensions

- Vues dynamiques (reflètent les modifications du dict) :
 - `d.keys()` : Parcourt les clés (comportement par défaut avec `for k in d:`).
 - `d.values()` : Parcourt les valeurs.
 - `d.items()` : Parcourt les paires `for cle, valeur in d.items():`.
- Bonne pratique d'itération : Ne jamais modifier la structure d'un dict (ajout/suppression) pendant son parcours direct ; itérer sur `list(d.keys())`.
- Compréhension de dictionnaire : `{cle: valeur for el in iterable if condition}`

Python

`inverse = {v: k for k, v in original.items()}`

6. Ensembles (set et frozenset)

Collections d'éléments uniques, non ordonnés et hachables.

Syntaxe et méthodes

- Création : `s = {1, 2, 3}` ou `set(iterable)`.
 - Attention : `{}` crée un dictionnaire vide. L'ensemble vide s'écrit obligatoirement `set()`.
- Ajout/Suppression :
 - `s.add(val)` : Ajoute un élément.
 - `s.discard(val)` : Supprime l'élément sans erreur s'il est absent.
 - `s.remove(val)` : Supprime l'élément, mais déclenche une `KeyError` s'il est absent.
 - `s.pop()` : Retire un élément arbitraire.
- Performance : Recherche `val in s` en temps constant $\mathcal{O}(1)$ (contre $\mathcal{O}(n)$ sur les listes).

Opérations ensemblistes

Opération	Opérateur	Méthode	Définition
Union	$a \mid b$	<code>a.union(b)</code>	Tous les éléments présents dans a ou b
Intersection	$a \& b$	<code>a.intersection(b)</code>	Éléments communs à a et b
Différence	$a - b$	<code>a.difference(b)</code>	Éléments dans a mais absents de b
Différence symétrique	$a \wedge b$	<code>a.symmetric_difference(b)</code>	Éléments dans a ou b, mais pas les deux

- Comparaisons : $a \leq b$ (sous-ensemble), `a.isdisjoint(b)` (aucun élément commun).
- `frozenset` : Version immuable du set. Fixé dès la création, il est hachable et peut servir de clé de dictionnaire ou d'élément dans un autre set.

7. Tuples et Namedtuples

Collections ordonnées et immuables délimitées par des parenthèses (). Plus légères en mémoire que les listes.

Propriétés et opérations

- Création : `t = (1, 2, 3)` ou sans parenthèses `t = 1, 2, 3`.
 - Tuple à un seul élément : La virgule finale est obligatoire : `(42,)`. Sans virgule, `(42)` est évalué comme un entier simple.
 - Conversion : `tuple(liste)`.
- Immuabilité : Toute assignation d'index `t[0] = x` déclenche un `TypeError`.
- Opérations supportées : Concaténation (`t1 + t2`), répétition (`t * 3`), `in`, `len()`, `min()`, `max()`, `.count(x)`, `.index(x)`.
- Déballage (Unpacking) :

Python

Affectation directe de valeurs multiples

```
somme, produit = calculs(5, 3)
```

`collections.namedtuple`

Sous-classe de tuple permettant d'accéder aux éléments par nom d'attribut tout en restant immuable et léger :

Python

```
from collections import namedtuple
```

```
Personne = namedtuple("Personne", ["nom", "prenom", "age"])
```

```
p = Personne("Dupont", "Alice", 30)
```

Accès par attribut ou index

```
print(p.nom, p[0])
```

Méthodes utilitaires :

```
d = p._asdict()          # Conversion en dictionnaire
```

```
p2 = p._replace(age=31)   # Crée une nouvelle instance modifiée
```

8. Slicing et Mécanismes de Copie

Applicable aux chaînes, listes et tuples via la syntaxe :

`sequence[start:stop:step]`

- start est inclus, stop est toujours exclu.
- Valeurs par défaut : `[:stop]` commence à 0, `[start:]` va jusqu'à la fin, `[:]` copie toute la séquence.
- Indices négatifs : Comptent à partir de la fin (-1 = dernier élément, -2 = avant-dernier).
- Pas négatif : `[::-1]` inverse intégralement la séquence (technique standard pour vérifier un palindrome : `mot == mot[::-1]`).
- Modification de liste par tranche : `l[1:3] = ["x", "y"]` remplace la plage ; `l[2:4] = []` supprime la plage.

Gestion de la mémoire : Shallow Copy vs Deep Copy

- Copie superficielle (shallow copy) : Réalisée par `seq[:]` ou `seq.copy()`. Crée un nouveau conteneur, mais conserve les références vers les objets internes mutables. Si une sous-liste est modifiée, l'original et la copie sont tous deux impactés.
- Copie profonde (deep copy) : Nécessite le module standard `copy`. Clone récursivement toute la structure et ses sous-objets pour éviter tout effet de bord :

Python

```
import copy
```

```
copie_profonde = copy.deepcopy(original_imbrique)
```

9. Expressions Régulières (Module re)

Recherche, validation et manipulation de motifs textuels.

Métasymboles et quantification

- Ancres : `^` (début de chaîne ou de ligne), `$` (fin), `\b` (frontière de mot), `\B` (non-frontière).
- Caractères génériques : `.` (tout caractère sauf saut de ligne `\n`), `|` (alternative OU).
- Ensembles [...]: `[abc]` (a, b ou c), `[a-z]` (plage), `[^0-9]` (exclusion de chiffres).
- Classes prédéfinies : `\d` (chiffre), `\D` (non-chiffre), `\w` (alphanumérique + `_`), `\W`, `\s` (espace, tabulation, saut de ligne), `\S`.
- Quantificateurs :
 - `*` : 0 ou plusieurs occurrences.
 - `+` : 1 ou plusieurs occurrences.
 - `?` : 0 ou 1 occurrence.
 - `{n}`, `{n,}`, `{n,m}` : Nombre précis ou encadré d'occurrences.
- Gloutonnerie : Les quantificateurs sont gourmands par défaut (capturent le maximum). Ajouter `?` pour les rendre non-gourmands/lazy (ex: `<.*?>` capture `<tag>` au lieu de `<tag>texte</tag>`).

Fonctions majeures du module re

- `re.search(motif, texte)` : Cherche la première correspondance n'importe où dans le texte.

- `re.match(motif, texte)` : Vérifie la correspondance uniquement dès le début de la chaîne.
- `re.fullmatch(motif, texte)` : Exige que le motif couvre l'intégralité du texte (idéal pour la validation stricte : codes postaux, identifiants).
- `re.findall(motif, texte)` : Retourne une liste de toutes les occurrences.
- `re.finditer(motif, texte)` : Retourne un itérateur d'objets Match.
- `re.sub(motif, repl, texte, [count])` : Remplace les motifs par une chaîne de substitution.
- `re.split(motif, texte)` : Découpe la chaîne selon les occurrences du motif.
- `re.compile(motif)` : Précompile l'expression régulière pour optimiser les performances lors d'appels répétés.

Groupes et Lookarounds

- Groupes capturants : Délimités par des parenthèses (). Numérotés de gauche à droite à partir de 1 (`m.group(0)` = motif complet, `m.group(1)` = premier groupe, `m.groups()` = tuple de tous les groupes).
- Groupes nommés : (`?P<nom>motif`), accessible via `m.group("nom")`.
- Groupes non capturants : (`?:motif`) pour appliquer un quantificateur sans stocker la capture.
- Lookahead (regard vers l'avant) :
 - Positif : (`?=...`) (vérifie que le motif est suivi par ... sans l'inclure).
 - Négatif : (`?!...`) (vérifie que le motif n'est pas suivi par ...).
- Lookbehind (regard vers l'arrière) :
 - Positif : (`?<=...`) (vérifie que le motif est précédé par ...).
 - Négatif : (`?<!...`) (vérifie que le motif n'est pas précédé par ...).

Drapeaux (Flags)

- `re.IGNORECASE` (ou `re.I`) : Rend la recherche insensible à la casse.
- `re.MULTILINE` (ou `re.M`) : Permet à `^` et `$` de correspondre au début et à la fin de chaque ligne.
- `re.DOTALL` (ou `re.S`) : Fait en sorte que le métasymbole point `.` corresponde également aux sauts de ligne `\n`.

Sécurité : Prévention du ReDoS (Backtracking Catastrophique)

Un déni de service par expression régulière (ReDoS) survient lorsqu'un motif avec des quantificateurs imbriqués ambigus (ex: (a+)+\$) analyse une entrée non conforme ("a" * 40 + "!"), provoquant un nombre exponentiel de retours en arrière qui fige le thread d'exécution.

- Contre-mesures : Bannir les quantificateurs imbriqués. Dès Python 3.11, employer des quantificateurs possessifs (a++, \d++) ou des groupes atomiques (?>...) qui désactivent le backtracking.

10. Structures de Contrôle : Conditions et Boucles

Branchements conditionnels (if, elif, else)

L'évaluation s'arrête à la première condition vraie.

Python

```
if note >= 90:
```

```
    mention = "Excellent"
```

```
elif note >= 80:
```

```
    mention = "Très bien"
```

```
else:
```

```
    mention = "Insuffisant"
```

Combinaison logique : and (conjonction), or (disjonction), not (négation)

```
if age >= 18 and not permis:
```

```
    statut = "Doit passer le permis"
```

Boucle bornée (for) et fonction range()

Itère sur n'importe quel itérable (chaînes, listes, tuples, dictionnaires).

- range(stop) : De 0 à stop - 1.
- range(start, stop[, step]) : De start à stop - 1 avec un pas défini.

Python

```
for i in range(0, 11, 2): # 0, 2, 4, 6, 8, 10
```

```
    pass
```

```
for lettre in "Python": # Itération caractère par caractère
```

pass

Boucle conditionnelle (while)

Répète un bloc tant que la condition reste évaluée à True. Nécessite une mise à jour d'état pour éviter les boucles infinies.

Python

```
compteur = 0
```

```
while compteur < 5:
```

```
    compteur += 1
```

Altération de flux de boucle

- **break** : Interrompt et quitte immédiatement la boucle courante.
- **continue** : Interrompt l'itération courante et passe directement à la suivante.
- **pass** : Instruction nulle (espace réservé syntaxique sans action).

11. Fonctions : Définition, Paramètres et Portée

Définies via le mot-clé `def`. Sans instruction `return` explicite, toute fonction renvoie implicitement `None`.

Paramètres et passage d'arguments

- **Paramètres positionnels et par défaut :**

Python

```
def saluer(nom, message="Bonjour"):
```

```
    return f"{message}, {nom}!"
```

- **Piège absolu des valeurs par défaut mutables** : L'argument par défaut est évalué une seule fois à la définition de la fonction. Un conteneur mutable (`list`, `dict`) sera partagé entre tous les appels.

Python

```
# INCORRECT : mémoire partagée
```

```
def ajouter(val, liste=[]): ...
```

```
# PATTERN IDIOMATIQUE
```

```
def ajouter(val, liste=None):
```

if liste is None:

 liste = []

liste.append(val)

return liste

Arguments variables (*args, **kwargs)

- ***args** : Capture les arguments positionnels supplémentaires dans un **tuple**.
- ****kwargs** : Capture les arguments nommés sous forme de **dictionnaire**.

Python

```
def log_requete(action, *nombres, **options):
```

```
    total = sum(nombres)
```

```
    debug = options.get("debug", False)
```

```
    return total
```

Fonctions imbriquées, Closures et Lambdas

- **Fonctions imbriquées** : Encapsulent une logique interne ou construisent des fonctions dynamiques.
- **Fonctions anonymes (lambda)** : Fonctions réduites à une seule expression retournée (idéales pour map(), filter(), sorted(key=...)).

Python

```
multiplier = lambda x, y: x * y
```

Récurtivité, Typage et Bonnes pratiques

- **Récurtivité** : Fonction qui s'appelle elle-même (nécessite obligatoirement un cas de base pour stopper la récursion).

Python

```
def factorielle(n):
```

```
    return 1 if n == 0 else n * factorielle(n - 1)
```

- **Type Hints et Docstrings (PEP 257)** :

Python

```
def calculer_moyenne(notes: list[float]) -> float:
```

```
    """Calcule la moyenne arithmétique d'une série de notes.
```

:param notes: Liste de notes numériques.

:return: Moyenne calculée.

"""

return sum(notes) / len(notes)

- **Pureté et isolation** : Éviter les effets de bord (ne pas modifier l'état extérieur ou des variables via global).

12. Gestion des Erreurs et Exceptions

Typologie des anomalies

- **Erreurs de syntaxe (SyntaxError)** : Échec de l'analyse du code avant son exécution (deux-points omis, parenthèses non fermées).
- **Exceptions à l'exécution** : Déclenchées pendant l'évaluation du programme (ZeroDivisionError, NameError, TypeError, ValueError, OSError, KeyError).

Structure complète try / except / else / finally

Python

try:

 valeur = int(donnee_brute)

 resultat = 10 / valeur

except ValueError:

 # Gère spécifiquement l'erreur de conversion

 print("La valeur n'est pas un entier valide.")

except (ZeroDivisionError, OverflowError) as err:

 # Capture multiple sous forme de tuple + assignation d'instance

 print(f"Erreur de calcul : {err} (args: {err.args})")

except Exception as err:

 # Exception générique (intercepte toute classe dérivée d'Exception)

 raise # Propage l'erreur à la pile supérieure sans la masquer

else:

 # Exécuté UNIQUEMENT si le bloc try s'est terminé sans aucune exception

```
enregistrer(resultat)
```

finally:

```
# Exécuté SYSTÉMATIQUEMENT (même en cas de break, continue, return ou crash)
```

```
nettoyer_ressources()
```

Règles de propagation et hiérarchie

- **Ordre des gestionnaires** : Une classe dans except intercepte aussi toutes ses sous-classes. Les exceptions dérivées doivent donc être déclarées **avant** leurs classes parentes (ex: placer except FileNotFoundError: avant except OSError:).
- **Déclenchement explicite (raise)** :

Python

```
raise ValueError("Donnée hors limite") # Instanciation explicite
```

```
raise ValueError          # Équivalent à raise ValueError()
```

- **Exceptions personnalisées** : Héritent obligatoirement de Exception.

Python

```
class ApplicationError(Exception):
```

```
    """Classe de base du module."""
```

```
    pass
```

```
class ValidationError(ApplicationError):
```

```
    def __init__(self, champ, message):
```

```
        super().__init__(f"Erreur sur {champ}: {message}")
```

```
        self.champ = champ
```

Gestionnaires de contexte (with)

Garantissent la fermeture et la libération déterministes des ressources externes (fichiers, sockets, locks) même en cas d'interruption ou d'exception, remplaçant les blocs finally manuels.

Python

```
# Gestion sécurisée de flux de fichier
```

```
with open("donnees.txt", "r", encoding="utf-8") as f:
```

```
for ligne in f:
```

```
    traiter(ligne)
```

```
# Le fichier est garanti fermé dès la sortie de l'indentation
```

13. Débogage et Analyse d'Erreurs

Anatomie d'un Traceback

Un traceback retrace la pile d'appels (*call stack*) au moment du plantage. Il se lit de **haut en bas** :

1. **Premières lignes** : Point d'entrée initial de l'exécution.
2. **Lignes intermédiaires** : Chaîne d'appels de fonctions (fichier, ligne, contexte).
3. **Dernière ligne** : Type de l'exception levée et message explicatif.

Plaintext

Traceback (most recent call last):

```
File "main.py", line 14, in <module>
```

```
    traiter_donnees(valeurs)
```

```
File "main.py", line 6, in traiter_donnees
```

```
    return total / len(donnees)
```

ZeroDivisionError: division by zero

Correction : Remonter à la ligne indiquée (main.py, ligne 6) et vérifier l'état des variables (donnees est vide).

Typologie des erreurs fréquentes

Exception / Erreur	Cause standard	Exemple	Correction type
SyntaxError	Code non analysable (oubli :, parenthèse non fermée).	if x == 2 print(x)	Corriger la syntaxe avant exécution.
NameError	Variable ou fonction appelée avant	print(total) sans déclaration	Vérifier l'orthographe et la portée.

Exception / Erreur	Cause standard	Exemple	Correction type
	affectation / faute de frappe.		
TypeError	Opération sur des types incompatibles.	"Age: " + 25	Transtyper explicitement (str(25)).
ValueError	Bon type, mais valeur inappropriée.	int("abc")	Valider la donnée en amont (str.isdigit()).
IndexError	Index hors des limites de la séquence.	l = [1]; print(l[2])	Vérifier len(l) ou utiliser une boucle for.
KeyError	Clé absente d'un dictionnaire.	d = {}; print(d["id"])	Utiliser d.get("id", default).

Erreurs de logique (Bugs silencieux)

Le programme s'exécute sans lever d'exception, mais produit un résultat erroné.

- **Erreurs de bornes (*off-by-one*)** : Confusion sur l'exclusion de l'indice de fin dans range(start, stop) ou le slicing.
- **Court-circuit booléen** : Mauvaise priorité entre and et or ; parenthéser systématiquement les conditions composées.
- **Mutation d'état partagé** : Modification involontaire d'une liste transmise par référence au lieu d'une copie.

Méthodologie d'investigation progressive

1. **Reproduction minimale** : Isoler le problème dans un script court sans dépendances superflues.
2. **Affichage ciblé** : Utiliser les f-strings auto-documentées pour inspecter les états intermédiaires :

Python

```
print(f"{variable=}, {type(variable)=}")
```

3. **Assertions défensives** : Bloquer l'exécution dès qu'une condition interne est violée :

Python

```
assert len(donnees) > 0, "La liste ne doit pas être vide"
```

4. **Débogueur interactif (breakpoint())** :

Insérer `breakpoint()` (Python 3.7+) dans le code pour suspendre l'exécution et ouvrir l'invite `pdb`.

- `n` (*next*) : Exécute la ligne suivante.
- `s` (*step*) : Entre dans la fonction appelée.
- `c` (*continue*) : Poursuit l'exécution jusqu'au prochain point d'arrêt.
- `p` variable (*print*) : Affiche la valeur d'une variable.
- `q` (*quit*) : Quitte le débogueur.